

# **Server-Side WebAssembly in Practice**

By Skaara Poncin, Storm van Loon, Gabriel Uwaila  
26 March 2026



# **Table of contents**

|                                      |           |
|--------------------------------------|-----------|
| <b>Table of contents</b>             | <b>1</b>  |
| <b>1. Abstract</b>                   | <b>2</b>  |
| <b>2. Introduction</b>               | <b>2</b>  |
| <b>3. How WebAssembly works</b>      | <b>3</b>  |
| <b>4. WASI</b>                       | <b>3</b>  |
| <b>5. Security</b>                   | <b>4</b>  |
| <b>6. Wasm for developers</b>        | <b>4</b>  |
| 6.1. Runtimes                        | 4         |
| 6.2. Language support                | 5         |
| 6.3. WebAssembly modules             | 6         |
| 6.3.1. Making Modules                | 6         |
| 6.3.2. Binary vs. text format        | 6         |
| 6.3.3. Imports and exports           | 6         |
| 6.4. WIT                             | 6         |
| 6.5. Component model                 | 7         |
| 6.6. Cross Language Interoperability | 8         |
| <b>7. HTTP servers</b>               | <b>9</b>  |
| <b>8. AI with wasm</b>               | <b>10</b> |
| <b>9. Wasm in the cloud</b>          | <b>11</b> |
| 9.1. Containers                      | 11        |
| 9.2. Pre-made hosts                  | 12        |
| 9.3. Kubernetes                      | 13        |
| 9.4. Edge                            | 14        |
| 9.4.1. CDN (CloudFlare)              | 14        |
| 9.4.2. Serverless                    | 14        |
| 9.4.3. Microprocessors               | 14        |
| 9.4.4. Wasm in databases (UDF)       | 14        |
| <b>10. Extra</b>                     | <b>15</b> |
| 10.1. WasmCon                        | 15        |
| 10.2. WasmIO                         | 15        |
| <b>11. Hype decline</b>              | <b>15</b> |
| <b>12. Future of wasm</b>            | <b>16</b> |
| 12.1. Confidential computing         | 16        |
| 12.2. AI Agents                      | 17        |
| 12.3. Edge computing                 | 17        |
| 12.4. WASI 0.3                       | 17        |
| <b>13. Conclusion</b>                | <b>17</b> |
| <b>14. Sources</b>                   | <b>18</b> |

# 1. Abstract

Server-side WebAssembly promises a portable, performant and sandboxed execution model that spans along development platforms, servers, edge-platforms and even databases. This paper investigates how this promise holds up in practice, by combining literature review with hands-on experiments in development, HTTP serving, AI inference, container alternatives and orchestration platforms. We first outline WebAssembly's core execution model, the role of WASI, the component model and then examine developer-facing concerns such as runtimes, language toolchains, WIT-based interfaces and cross-language interoperability. Building on this, we implement HTTP components in Rust, Go and C++, prototype AI workloads compiled to wasm, while also exploring deployment on Docker, Kubernetes, WasmCloud, Spin/SpinKube, CDN edges, microcontroller runtimes and database UDF engines. Our evaluation confirms major benefits around cold starts, file size and cross-language usage. However, it also surfaces significant gaps in documentation, uneven language and runtime support, immature integration with major players and lackluster adoption of WASI preview 3 features. We conclude that server-side WebAssembly is transitioning from hype to a more modest, albeit growing role, with the clearest near-term value in edge and FaaS platforms, plugin-style extensibility and security-sensitive workloads like confidential computing and AI agents.

# 2. Introduction

The web has run on JavaScript for decades. While this technology was groundbreaking at the time, its age has started to show: JavaScript's inefficiency and closed-offness limited innovation in the web. WebAssembly was made by the four major browser vendors, unveiled in 2015 and released in 2017, and specifically designed to solve these problems; it sought to improve web performance, while staying compact, portable and secure. (Haas et al., 2017, 1)

WebAssembly – wasm for short – allows code written in languages like Rust, C or Go to run directly in the browser. Instead of being interpreted like JavaScript, WebAssembly is compiled into a compact binary format that browsers can execute very efficiently. This enables developers to bring performance-intensive applications, like games, video editing or streaming, to the web while maintaining near-native speed. Real-world examples include in-browser Google Earth (Mears, 2019), Figma (Andersson & Veteto, 2017) or even Photoshop (*Photoshop's Journey to the Web | Articles*, 2021).

Coincidentally, the focus on portability – meaning that WebAssembly should run with every browser on every device – meant that wasm was not just limited to browsers. Specific runtimes (more on that later) were made to run wasm apps on the server-side. Promises specifically for the server-side were based on the original goals for WebAssembly – meaning performance, portability, security and compactness – but were more aimed towards cross-language interoperability, sandboxed environments, and hardware agnosticism.

In this paper, we will discuss the different applications of server-side WebAssembly, their marketed performance and whether support is satisfactory.

### 3. How WebAssembly works

Before going into the practical explanation, it is important to get the background on how WebAssembly is actually made and why it's more performant than alternatives. The model is based on assembly-like code. Thus, the memory model is very simple; it only has two fundamental elements: Linear memory and four primitive data types (int32, int64, float32 and float64). More complex data types, e.g. strings, are translated by the compiler of a specific language to two integers: The first is a pointer to where the string starts – an array of encoded bytes in UTF-8 format, while the second is the length of the string.

To increase support for other languages, an interface was built specifically for translating complex data types to wasm types – see the section on WIT (section 6.5).

Due to its platform agnosticism, WebAssembly apps run on a host – a runtime of some sorts, and include a .wasm file. This wasm file consists of a **host application**, and can include a **guest application**.

The host application runs the entrypoint and orchestration logic. This can be either compiled to WebAssembly, or run in its own programming language. On the web, the host application is the JavaScript file using wasm, while on the server-side, this can be in a variety of different languages (see [section 6.6](#)).

The guest refers to dependent modules or components – they can be compared to libraries or dependencies. These are always compiled to WebAssembly.

Wasm apps are run with a runtime – either the JavaScript engine on the web, or dedicated runtimes (see section 6.1). Wasm modules are not only apps, but also share similarities with containers – see the section on containers (section 9.1).

### 4. WASI

As mentioned in the previous section, WebAssembly is platform agnostic – so interacting with the underlying system is not straightforward. Features like saving data are not built into wasm as a format, but must be provided by the host environment. The WebAssembly System Interface – or WASI – is a “group of standards-track API specifications for software compiled to the W3C WebAssembly (Wasm) standard. WASI is designed to provide a secure standard interface for applications that can be compiled to Wasm from any language, and that may run anywhere—from browsers to clouds to embedded devices.” (WASI Subgroup, n.d.). In simple terms, the interface makes features like HTTP servers, timers or the **Component Model** (see [section 6.5](#)) possible.

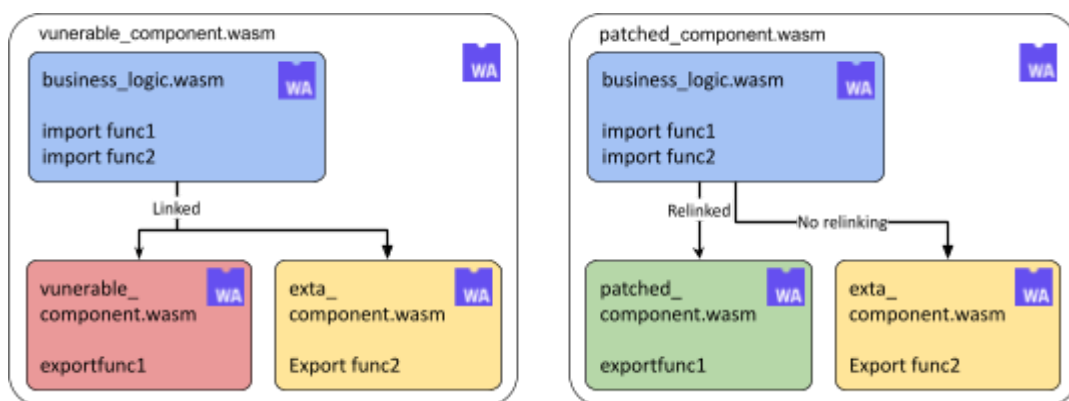
WASI has multiple versions, ranging from 0.1 to 0.3. While this is the official versioning, WASI is usually referenced to as WASI *preview* one, two or three. This will be most obvious

in compiling wasm apps, where you often compile to wasip2. WASI preview 3 has been released in early 2026, yet adoption is up to runtimes *and* language tooling.

## 5. Security

Security is and will continue to be a core part of WebAssembly modules, although there are also downsides with using wasm. Wasm modules are more secure due to it running in a memory-safe, sandboxed environment, while also needing explicit imports for system functions.

Wasm *components* are even more secure, due to its **shared-nothing security**. These components do not share a memory space and thus, are even better sandboxed than modules. Additionally, components can only access specified imports. If a library, used for writing data – e.g. saving a pdf – is compromised and now also reads data, the app will still only be able to access the permissions given by the developer. Finally, vulnerability mitigation is improved over alternative programs, as wasm components – read, dependencies – can be switched out without rebuilding the entire program.



A security downside to WebAssembly is its unreadability. Wasm code is binary code and thus, more difficult to analyze for malware. Developers for WebAssembly did take this into account and created the .wat file format; a readable version of their binary code. To compare this yourself, see section 6.3.2.

## 6. Wasm for developers

### 6.1. Runtimes

WebAssembly was created specifically for browsers, which come with a wasm engine already embedded. Thus, running wasm on normal operating systems needs an additional step: WebAssembly runtimes. These make it possible to run this technology outside of the browser; on any server, desktop or even IoT devices.

People with a Java background might recognize similarities between wasm runtimes and the Java Runtime Environment. The wasm runtime is the execution environment that loads, validates, compiles and runs a .wasm module (KodeKloud, n.d.). There are four main steps that these runtimes do:

- Load the wasm module
- Verify the AOT wasm binary
- Compile wasm binary to JI machine code – most commonly with a Cranelift or LLVM backend
- Run the machine code inside the sandbox environment

Additionally, they have other uses like mediating calls between wasm modules and host APIs (KodeKloud, n.d.), garbage collection, etc.

Picking a runtime for a specific use case can be challenging. In this research, we selected a few specific runtimes – which will be explained in more detail later. However, below is a list of other common choices.

| Runtimes  | js / web | java | IOT devices | server / general | education |
|-----------|----------|------|-------------|------------------|-----------|
| node.js   | ✓        |      |             |                  |           |
| deno      | ✓        |      |             |                  |           |
| GraalWasm |          | ✓    |             |                  |           |
| Chicory   |          | ✓    |             |                  |           |
| Wasm3     |          |      | ✓           |                  |           |
| WAMR      |          |      | ✓           |                  |           |
| Wasmtime  |          |      |             | ✓                |           |
| Wasmer    |          |      |             | ✓                |           |
| WasmEdge  |          |      |             | ✓                |           |
| Wazero    |          |      |             | ✓                |           |
| Wizard    |          |      |             |                  | ✓         |

## 6.2. Language support

Compiling languages to wasm or WASI is fully dependent on the language itself; they have to create the tooling to compile to WebAssembly. For example, while Rust has the wasm32-wasip2 compiler target built into the standard Rust installation (with full up-to-date support), other languages like Go only have limited support, e.g. only the wasip1 target, and a third-party compiler called TinyGo with less, but still limited wasm support.

## 6.3. WebAssembly modules

The explanation for wasm modules was already given in section 3, yet there are a few significant remarks for developers specifically.

### 6.3.1. Making Modules

Making modules or components ([6.5 Component model](#)) themselves is easy, as long as the language has compiler target support for your WASI needs – see section [6.2 Language support](#). However, there are notable exceptions.

Python is an interpreted language – which means that it does not compile to a native binary, like Rust or C do. That being said, it is possible to make module apps using CPython, yet it is not possible to make a guest module to use within a host.

While it's not possible to make a guest *module*, it is possible to make python guest *components* by bundling the entire CPython interpreter into a wasm binary, with a tool called componentize-py. Again, look at section [6.5 Component model](#) for more component explanation.

### 6.3.2. Binary vs. text format

Wasm modules can exist in two formats. The binary format .wasm is what runtimes actually use to execute. The text format .wat (WebAssembly Text Format) is the human readable version of .wasm – making it easier to inspect, debug or even write by hand for the dedicated. Developers can use tools like wat2wasm (<https://webassembly.github.io/wabt/demo/wat2wasm/index.html>) and wasm2wat from WABT to convert between the two.

### 6.3.3. Imports and exports

Modules communicate with their host environment through declared imports and exports. Imports can be functions or memory – everything the module needs from the host before it can run. Exports are the connections that make modules available to the host environment. This strict contract enforces the sandbox. Nothing can pass unless it was declared. This is more rigid compared to the WIT based approach with component models, but also less complex and more broadly supported over runtimes and languages.

## 6.4. WIT

As mentioned previously, wasm modules on their own don't have any access to underlying system functions or complex data types. While these can be programmatically created by the developers themselves, the **WebAssembly Interface Types** or WIT solves this problem.

WIT is an interface built on the core of WebAssembly – meaning linear memory and primitive data types. As it's an interface definition language, WIT itself is program language agnostic and thus, can be used with any language.

The interface adds various types: booleans, signed and unsigned integers and floats of various sizes, characters, strings, lists, even optional values. Besides types, error handling through results (result<string, string>), tuples for grouping values, records, enums and enum-variants that hold data, flags for bitwise combinations of options (read, write, execute) *and* aliases for existing types and functions are also possible with WIT.

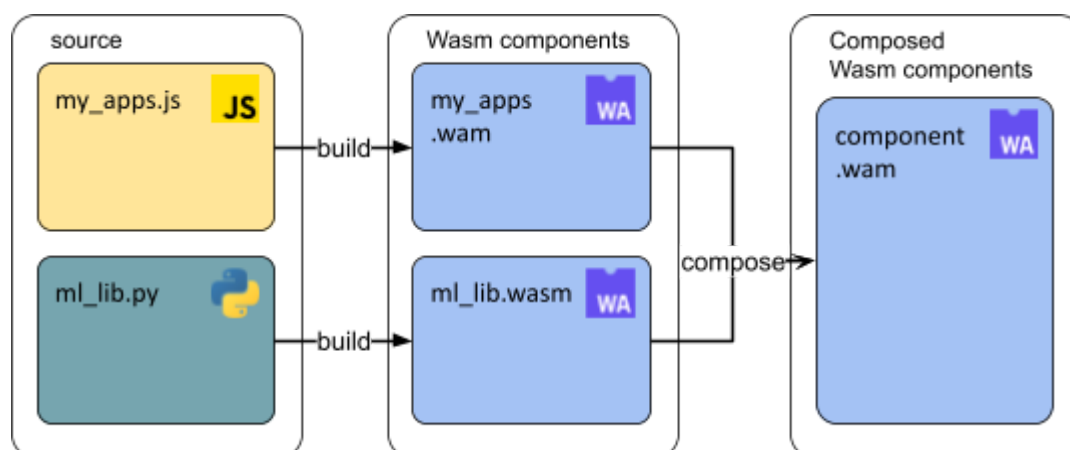
The list given is very extensive, so it raises the question – how does it work? WIT standardizes how to convert from other languages to WebAssembly via the **canonical ABI** – application binary interface. Another advantage of WIT is that both low- and high-level languages can be used together with the interface.

To see examples of how WIT files are used, check out the GitLab project linked to this research ([https://gitlab.com/kdg-ti/the-lab/teams-25-26/44\\_thetinkerers](https://gitlab.com/kdg-ti/the-lab/teams-25-26/44_thetinkerers)).

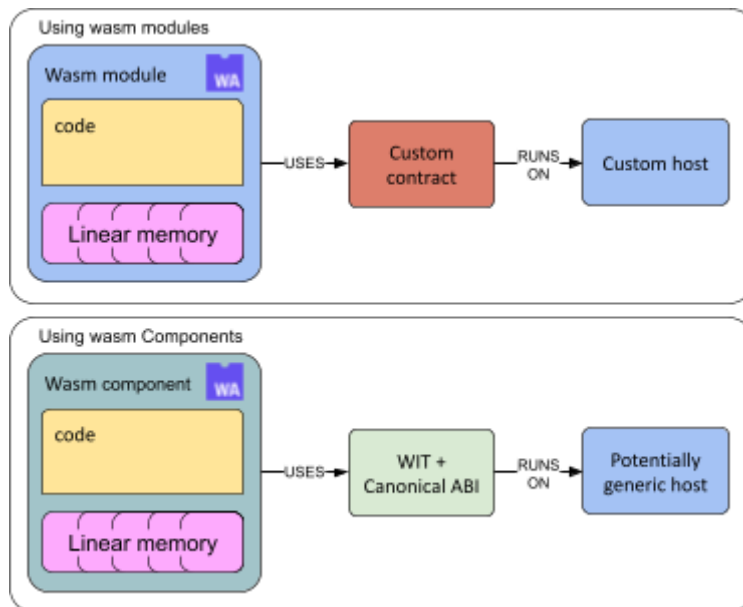
## 6.5. Component model

While modules have their own benefits (e.g. more support), components offer a list of different advantages. The component model makes it very simple for wasm modules to interact with the underlying system – given that the necessary permissions were configured. This means that, if supported, components can support vastly different programs than modules were able to.

Besides added features, the **component model** offers more portability and security than modules would. Component security has been explained already (see section [5. Security](#)), yet the portability is more important to developers specifically. If a library and host both get compiled to a wasm **component**, the two components can be combined to one **composed** wasm component. As mentioned in the security section, the two components do not share memory, but use the other components' function calls. Additionally, this composed component can be used in other applications.



Using components also means that a component does not need a specific host, but can use a generic host. WIT, combined with the canonical ABI allows developers to use this standardized contract, without having to write specific code for that component specifically.



## 6.6. Cross Language Interoperability

With all of the surrounding information explained, one of the core reasons to use WebAssembly can be handled: compiling other languages into wasm/WASI and running them inside another language. The usual approach to cross language libraries often have some limitations, like only working on a certain OS (e.g. .dll files) or needing new bindings for different languages. Wasm solves this, compile it once and reuse it in every other language you want, regardless of OS or machine architecture.

The biggest advantage within this functionality is the sandbox environment: a Wasm module has no access to the host machine unless explicitly granted. This applies to dependencies as well. That means a third party library can not read or write the filesystem, unlike npm or pip packages. Besides, each module has its own memory space – meaning that a bug or vulnerability in any module will not compromise any piece of memory outside of that module.

Next to the benefits of security, Wasm modules also make it possible to hot swap modules without editing or rebuilding the host application. This opens the option for other developers to create plugins in any language of choice, which the host can safely load in a sandbox, without any risk of the application.

To run any Wasm module or component, the language needs to support a wasm runtime. This is where support varies drastically, based on language. For this part, any runtime can be chosen, but here are some options that we looked into.

- Rust is seemingly the best testing platform for this wasm/WASI: it has Wasmtime as a library, with full wasip2 support. Additionally, wasmtime is written in Rust itself.
- C and C++ are supported by wasmtime, with C++ specifically having “a header-based library layered on top of the C API” (wasmtime, n.d.)
- Go bindings are used through wasmtime-go using CGO
- Python once again uses wasmtime as a package on PyPi
- Java can use GraalWasm

The table below specifies which language can be used on which runtime as the host application, with extra information on whether or not the language supports module or component support as a guest.

| language | host        | guest module | guest component |
|----------|-------------|--------------|-----------------|
| Rust     | wasmtime    | ✓            | ✓               |
| C/C++    | wasmtime    | ✓            | ✓               |
| GO       | wasmtime-go | ✓            | ✓               |
| Python   | wasmtime    | ✗            | ✓               |
| JS       | Node.js     | ✓            | ✓               |
| Java     | GraalWasm   | ✗            | ✗               |

## 7. HTTP servers

This section describes a real-world example, to analyze how server-side WebAssembly works in a realistic web scenario.

There are multiple reasons on why wasm might be used for HTTP servers. First off, is portability: the same .wasm file can run on any supported host without recompiling – the core promise of build once, deploy everywhere.

Secondly – as discussed earlier – these applications are more secure and sandboxed. Unless specific permissions were set up, there is no access to any filesystem, OS or network – besides the necessities WASI set up for HTTP to work.

The third major benefit of wasm-based HTTP applications, are the extremely fast cold starts. Runtimes like wasmtime, combined with Spin (see [section 9.2](#)) can fire up many instances with extremely low overhead – two major benefits for serverless and edge platforms (see [section 9.4](#)).

The application itself was tested with three server-side wasm components that all implement the same library: “wasi:http/incoming-handler”. This returns a “hello-world” response. In our testing, three different languages/toolchains were used; Rust, TinyGo and C++, all with wasmtime serve.

The main differences are highlighted below, yet the main insight is consistency across languages. After all, the target is wasip2.

Important to note is that C++ had the highest development friction, mainly due to ‘wit-bindgen c’. This emits C symbols, so the program must wrap with ‘extern "C"’ when including from C++, or name mangling breaks the linker and ‘hello.c’ must be compiled with the C compiler and not C++ for the same reason.

Below is a table describing differences and/or similarities between three different languages implementing the same hello-world http example.

|                      | Rust                                          | TinyGo                                           | C++                                                  |
|----------------------|-----------------------------------------------|--------------------------------------------------|------------------------------------------------------|
| Binding tool         | wit_bindgen::generate!() (compile-time macro) | wasi-go-sdk (wraps raw WIT bindings)             | wit-bindgen c (manual codegen step)                  |
| Target               | wasm32-wasip2 (built in to Rustc)             | tinygo -target=wasip2                            | WASI SDK<br>wasm32-wasip2-clang++                    |
| Entry point          | export!(HttpServer) — no main()               | init() + empty main()                            | extern "C" void exports_...handle(...) — no main()   |
| API style            | High-level Rust types + trait                 | Familiar http.Handler style                      | Raw C structs and handles                            |
| Toolchain setup      | Rustup target add<br>wasm32-wasip2            | brew install tinygo + Go<br>1.19–1.25            | Download WASI SDK +<br>cargo install wit-bindgen-cli |
| Build command        | cargo build --target<br>wasm32-wasip2         | tinygo build -target=wasip2<br>--wit-package ... | 3-step: codegen → compile<br>C → compile C++ → link  |
| Run command          | wasmtime serve -Scli *.wasm<br>same<br>same   |                                                  |                                                      |
| Development friction | Low                                           | Medium                                           | High                                                 |

Per GitLab: (Uwaila, 2026)

## 8. AI with wasm

Nowadays, artificial intelligence is a core part of server-side technologies. WebAssembly has also been implemented in this space.

Instead of treating AI as a service that must live in a separate Python environment, we experimented with moving the heavy inference logic into Rust and compiling that into wasm – similar to Colin Murphy’s work. Murphy – as a demonstration – rewrote an AI embedding software in Rust, with a preloaded model (model.onnx). This model would be loaded once and cached afterwards, so only the first request pays a slight compilation cost, yet the other requests run in the order of seconds. Besides ONNX, there are other machine learning SDKs supported by wasmtime, like OpenVINO and WinML.

These preloaded models can be combined with WebAssembly-based servers. Frameworks like Spin simplify this by handling deployment and execution details. In a real-world environment, the flow starts with a user triggering a request (e.g. by clicking a button), which runs a wasm module – with the AI model – using Wasmtime and Spin. This execution is very fast – typically milliseconds to about a second – and the process terminates immediately after completing its task.

WebAssembly-powered AI does not stop at this simple example. Other examples include but are certainly not limited to the following:

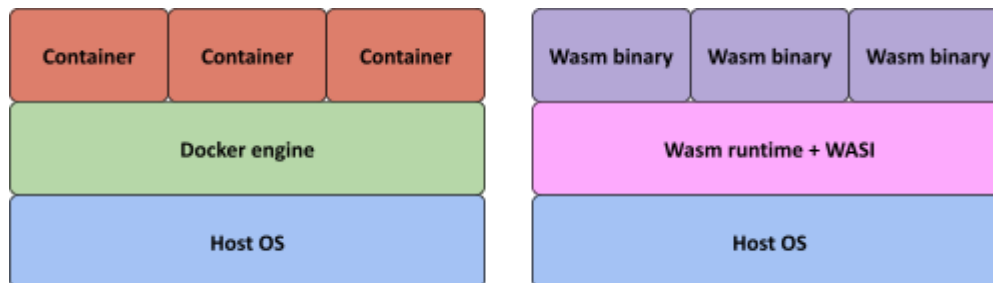
- Visual search: Top-k similar images, e.g. “search by image”, similar stock photos.
- Recommendations: “Users who viewed this also liked...” via embedding proximity.

- Tagging / clustering: Group by embedding distance; suggest or assign categories at the edge.
- Deduplication / filtering: Detect near-duplicates; hide or merge.
- Downstream ML / analytics: Store embeddings and run offline models (anomaly detection, quality scoring, trends) in the cloud later.

## 9. Wasm in the cloud

### 9.1. Containers

WebAssembly can, at times, be similar to containers. They can also package all dependencies and if deployed correctly, have an abstraction layer. See the illustration for their comparison. These wasm workloads are referred to as wasm **binaries**. These binaries have no need for an underlying OS; they only need a runtime for the applications to run.



At the peak of their hype, wasm ‘containers’ were supported by docker. The co-founder of docker even mentioned that “docker wouldn’t have been created if web assembly existed”. Docker supported a wasm-specific ‘shim’ to make it possible for wasm containers to run without loading the unnecessary dependencies Docker usually loads for containers.

This did not mean that docker containers were no longer relevant. Whenever lots of system calls were needed, it was still better to use docker containers – WASI still does not allow for all system functions.

While this support hasn’t faded entirely, as of March 2026, Docker Desktop support is deprecated and only guesses can be made for if and how long Docker Engine support will continue for. There are no official statements on why Docker support is waning.

Even if support is reduced in the future, the implementation of wasm binaries still has multiple advantages to using docker containers. These are the following: (KodeKloud, 2023)

- Smaller containers (no userspace): up to 90% smaller
- Faster startup times: up to 1000x faster
- Platform agnostic (portability is dependent on runtime developers). Note that while docker images are *not* platform independent, wasm binaries *are*.
- Secure by default

A notable vendor is WasmEdge – although it should be kept in mind that docker integration of wasm is moving away.

## 9.2. Pre-made hosts

In the developer section, running wasm was mostly done with custom hosts. However, it's also possible to just deploy wasm apps to pre-made hosts like WasmCloud and Spin. Both of these have wasmtime embedded, and are solutions focused on the deployment side. While some configuration is still necessary, this cuts down on development time and offers additional features, like scaling and Kubernetes-integration.

Wasm apps use WIT for their dependencies (see section 6.4. WIT). This does not change with these providers. The configuration of your host is done through a .yaml file – similar to Kubernetes. Running an HTTP server on WasmCloud is more complex than it might seem. It consists of the WasmCloud Host, the NATS lattice and wadm monitor.

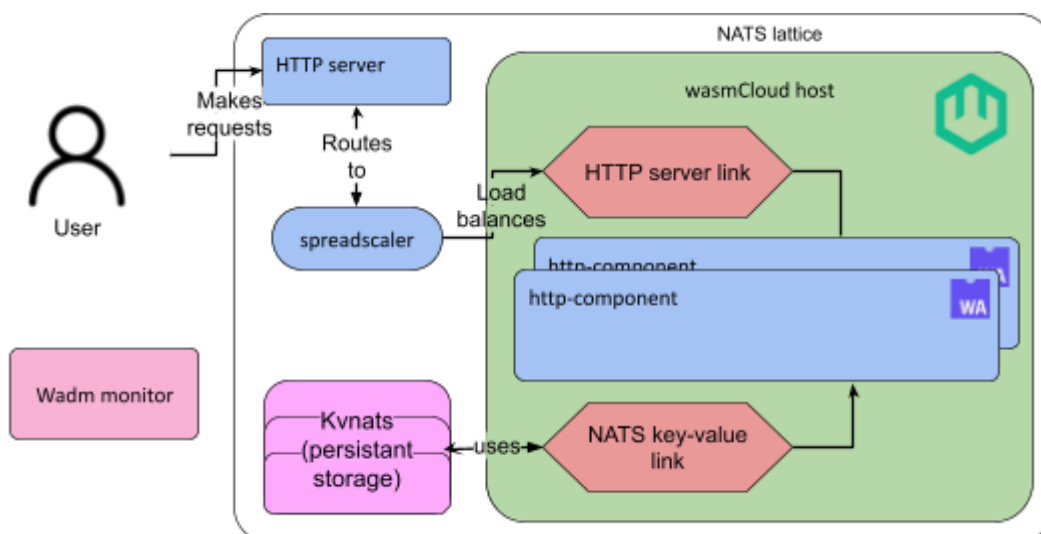
- The host is configured with the .yaml file, and executes the wasm modules or components.
- The NATS lattice sets up a NATS server. This lets multiple hosts communicate with each other, even on different machines. This can be useful for scaling.
- The wadm monitor monitors the lattice and manages the state of deployments. Hosts must be linked here.

WasmCloud is similar to Kubernetes in some ways:

- It orchestrates the running of multiple wasm instances,
- Has self-healing
- Can scale deployments on different machines

An example program with a persistence layer on WasmCloud can be found on our GitLab project:

[https://GitLab.com/kdg-ti/the-lab/teams-25-26/44\\_thetinkerers/demo/-/tree/main/WasmCloud-persistence](https://GitLab.com/kdg-ti/the-lab/teams-25-26/44_thetinkerers/demo/-/tree/main/WasmCloud-persistence).



Besides WasmCloud, another often-used host is Spin. Each solution has their own target audience. While WasmCloud has a scaler and can support third-party implementations, Spin has more language support and offers pre-made templates for HTTP, Redis and even a file server in multiple languages.

### 9.3. Kubernetes

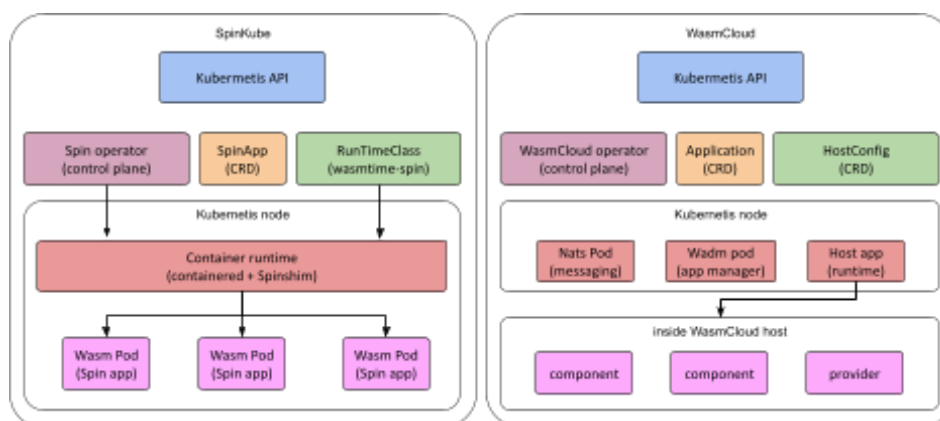
With docker support going away, it was necessary to look for alternatives. Lots of branches of the wasm-community advertise Kubernetes integration, yet implementation differs greatly from the advertisements. While it was advertised that stand-alone runtimes like wasmtime, wasmer and wasmedge would work, this unfortunately was not the case without additional configuration. To see the steps undertaken, visit its respective GitLab project:

[https://GitLab.com/kdg-ti/the-lab/teams-25-26/44\\_thetinkerers/containers](https://GitLab.com/kdg-ti/the-lab/teams-25-26/44_thetinkerers/containers)

Alternatives like k3d and k Rustlet are advertised to work, yet have their own issues. K3d is dependent on docker support (k3d, 2024) and thus, can't be relied on for wasm binaries. K Rustlet on the other hand is made specifically for wasm, yet hasn't seen updates in years (K Rustlet, 2023).

For this research, two other alternatives were considered: WasmCloud and SpinKube (the Kubernetes integration of Spin). However, due to the limited time, we unfortunately could not test whether they do work. They advertise to use Custom Resource Definition to make things other than containers, run with the Kubernetes API. Note that these solutions have their own overhead and will be less efficient than running wasm binaries without these frameworks.

WasmCloud deploys the full environment inside the cluster, while SpinKube needs a cluster with key components like a certificate manager, runtime class, CRDs and operator from SpinKube. To debug WasmCloud, the wadm tools are used. Meanwhile, SpinKube uses the integrated Kubernetes tools.



## 9.4. Edge

Edge computing runs code close to users on CDN networks, serverless platforms and even tiny microprocessors. This section looks at WebAssembly on CDN edges, serverless functions and constrained IoT devices.

### 9.4.1. CDN (CloudFlare)

Traditional CDNs only cached static assets, but modern CDNs like Cloudflare Workers, Fastly Compute and Akamai's Spin-based platform now execute WebAssembly at the edge. These platforms compile Rust, Go or C/C++ to Wasm and use WASI to provide a portable interface for HTTP and other functionalities, while keeping the code sandboxed. Cold starts can be sub-millisecond and memory footprints are a few megabytes, which makes running per-request logic (e.g. personalization or filtering) feasible on hundreds or thousands of devices worldwide. Akamai's acquisition of Fermion (Spin) extends this model to more than 4000 edge locations, combining Spin's Wasm serverless model with Akamai's existing CDN footprint.

### 9.4.2. Serverless

Many serverless platforms now support WebAssembly as a function runtime, rather than only JavaScript or containers. Cloudflare Workers and Fastly Compute run Wasm modules or components per request with strict CPU and memory limits, while hosts proxy network and storage access via WASI, which keeps functions portable and sandboxed. In practice, this is a natural deployment target for the HTTP components we built in Rust, TinyGo and C++: the same "wasi:http/incoming-handler" interface can be hosted on Workers, Compute or Spin without recompiling. Akamai's Spin integration even goes one step further by treating Spin components as FaaS on its edge network, aiming for basically no cold start behavior at global scale (Wright, 2025).

### 9.4.3. Microprocessors

WebAssembly is not limited to just servers. Its very compact size makes it possible to run wasm on very small IoT devices. While native is still most efficient, the argument for portability can outweigh the performance advantage. There are two main runtimes for microprocessors: **WAMR** (or WebAssembly Micro Runtime) and **wasm3**. WAMR is slower than their alternative, yet provides features like AOT compilation, multithreading, OS integration (e.g. with Zephyr) and modularity. (Has, 2025)

### 9.4.4. Wasm in databases (UDF)

Running wasm on the edge is possible due to their compact size and improved start times. Besides these advantages, wasm is also sandboxed. This means that with **user-defined functions** (or UDFs), wasm can be embedded inside of the database itself. These UDFs are custom functions that extend the database engine. Usually, these are written in C or SQL, but wasm support means that any language can be used.

Running wasm in the database itself reduces the latency and improves performance for computation-heavy calculations – for example, an encrypt/decrypt function. If this happens

outside of the database, the networking between the database and calculation app vastly slow down the process. To see how this works in reality, visit an example on the WasmEdge page: <https://wasmedge.org/docs/embed/use-case/libsql/>.

## 10. Extra

Staying in touch with Wasm implementations can be difficult. However, there are two noteworthy conferences that focus specifically on WebAssembly that make it easier to see where WebAssembly is being implemented, and what trajectory it will take in the future.

### 10.1. WasmCon

WasmCon is an annual conference, held around the world. The upcoming conference will be held on 23 March 2026 in Amsterdam. One highlighted project was also applied in this paper: GraalWasm.

In short – GraalWasm makes it possible to run wasm modules in Java.

Another noteworthy mention is Hyperlight-Wasm; This might prove to be an alternative for running wasm workloads natively. Other speakers focus more on the client-side of WebAssembly.

### 10.2. WasmIO

Another honorable mention is WasmIO, a conference held yearly around March. The first WasmIO conference was held in Barcelona, Spain, on March 23-24, 2023. This is a two day single track conference focused on the wasm ecosystem (WasmIO, 2025).

A key focus of these conferences are the panel talk and demo videos on running AI workloads with wasm, and especially the emphasis on using Wasm as a portable abstraction layer for AI inference across browsers, edge devices and Kubernetes clusters. Speakers showed that AI models compiled to Wasm – often via ONNX – can run with near-native performance, fast cold start and strong sandboxing, making them suitable for cost efficient inference at the edge.

## 11. Hype decline

Early hype around server-side WebAssembly was presented as a potential replacement for the JVM, containers and microservices, but developer surveys show criticism about the maturity of the technology and its documentation. Public discussions from long time wasm contributors describe how big companies bet on WASI – even beyond wasip1 – and yet the component model did not translate into wide adoption. The market cooled down, many startups were acqui-hired and only teams using wasm for real life production/applications remain (Akbar, 2026), like Adobe, Figma, etc. Commenters in threads argue that Wasm's huge potential on paper dwarfs its actual wins, while there is slow, yet visible progress. Technical gaps reinforce this, since proposals like WASI SQL have seen little improvement and non browser APIs and tooling remain incomplete, so wasm rarely displaces mature stacks like docker containers and such.

Despite the hype-decline over the years, work such as WASI 0.3's asynchronous I/O continues (Byteiota, 2026) and server-side WebAssembly now grows more quietly in niches like edge/FaaS and internal platforms rather than a wholesale replacement for existing runtimes.

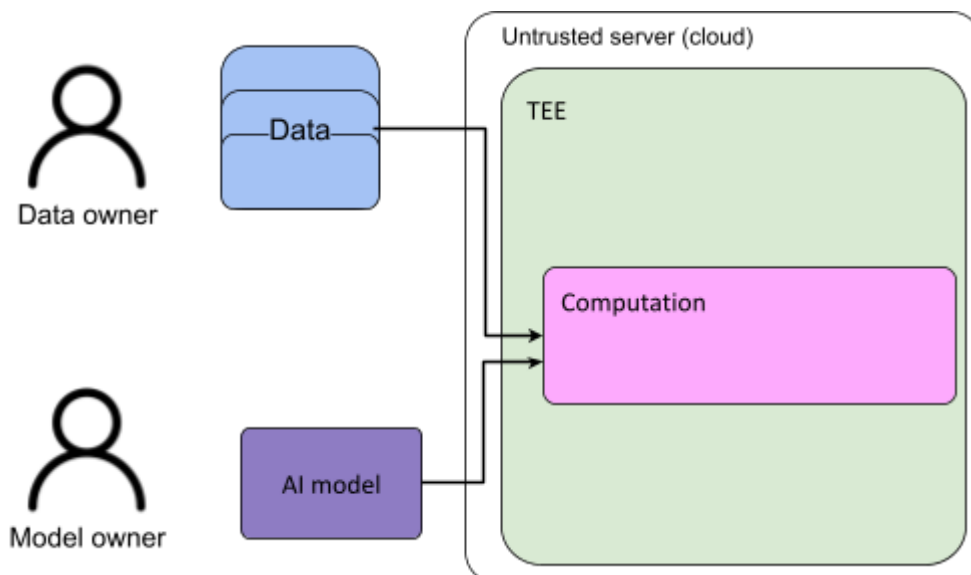
## 12. Future of wasm

While the future of WebAssembly on the server is difficult to predict, there are a few implementations where wasm is *the* solution for.

### 12.1. Confidential computing

The key for handling data is to encrypt data. Not just in transit with TLS, but also at rest, with hashing algorithms like SHA. But what about data in use? This question is becoming increasingly important as compute is often offloaded to major cloud vendors. In theory, this means that if someone reads the memory of these machines, the data can be extracted.

A solution for this is the **Trusted execution environment**: the TEE or enclave. This uses CPU extensions to encrypt memory access, so it's not accessible from outside of the enclave. Most CPU vendors already support this feature. With its current implementation, the traditional linux OS means a more complex deployment with more potential vulnerabilities. Additionally, these extensions are often written in C or C++. Wasm can solve both problems, with a secure, sandboxed environment and its cross-language possibilities.



An example of how this would be used in practice is **secure multiparty collaboration**: a proprietary model that should be applied on sensitive data, e.g. you have a proprietary AI model, yet the hospital can't share patient records. By running both the model and data in the TEE, neither party can access it. To see a company use the TEE, visit [CoCoS.ai](https://CoCoS.ai).

## 12.2. AI Agents

As mentioned in its dedicated section, machine learning is becoming an increasingly important part in our lives. AI agents specifically, are popping up everywhere. They often have access to APIs and thus, have an increased chance of prompt injection. With wasm, system features have to be explicitly allowed and additionally, the program is sandboxed either way. So even if something goes wrong, there are only local problems.

## 12.3. Edge computing

While this subsection also has its own dedicated part, we found no public evidence of major CDN vendors using wasm in this specific way yet. A dynamic website with a forum often uses a WAF, or Web Application Firewall. This can filter out bad content or even monitor for malicious prompts, e.g. SQL injections. If the WAF could be deployed at the edge, this process can cut down wait times significantly.

A proof of concept can be found here: <https://github.com/corazawaf/coraza-proxy-wasm>

## 12.4. WASI 0.3

While WASI 0.3 has been released already, implementation lacks severely. It's up to the runtimes and languages to implement these features, yet the improvements over WASI 0.2 are undoubtedly significant. Features include, but are not limited to support for asynchronous functions, cancellation, stream optimization (such as splicing and skipping) and even support for threads. To stay in touch with the progress of WASI and implementations, watch out for the roadmap: <https://wasi.dev/roadmap>.

# 13. Conclusion

On paper, server-side WebAssembly can be a compelling option in both application development and server operations. In practice, documentation, adoption and implementation lack severely, blocking its potential progress. The major benefits of using WebAssembly currently are very dependent on adoption of major players – the programming languages, major cloud vendors and container orchestration tooling.

Currently, a few aspects of the ecosystem work – albeit with inconsistent or lacking documentation. Cross-language interoperability is currently a major benefit for developers, while wasm-based HTTP deployments revolutionize cold starts in the server world.

The ecosystem is still expanding, as can be seen in the oncoming product releases, such as GraalWasm, or Azure's Hyperlight-Wasm. On the other hand, private development of these technologies (e.g. Akamai) will still improve the ecosystem altogether.

While its initial hype has undoubtedly passed, the server-side WebAssembly environment is still growing, with many implementations on its way.

## 14. Sources

### References

- Akbary, S. (2026, January 9). *What happened to WebAssembly*. Hacker News. Retrieved March 17, 2026, from <https://news.ycombinator.com/item?id=46551044#46551317>
- Andersson, R., & Veteto, V. (2017, June 8). *Figma is powered by WebAssembly*. Figma. <https://www.figma.com/blog/webassembly-cut-figmas-load-time-by-3x/>
- bytecodealliance. (n.d.). *Using the Wasmtime API*. Using the Wasmtime API. Retrieved March 16, 2026, from <https://docs.wasmtime.dev/lang.html#officially-supported-bindings>
- BytecodeAlliance. (n.d.). *Wasmtime - A fast and secure runtime for WebAssembly*. Wasmtime. Retrieved February 6, 2026, from <https://wasmtime.dev/>
- bytecodealliance. (2017, August 29). *wasmtime*. wasmtime. Retrieved March 16, 2026, from <https://github.com/bytecodealliance/wasmtime>
- bytecodealliance. (2020, April 6). *wasmtime-go*. wasmtime-go. Retrieved March 16, 2026, from <https://github.com/bytecodealliance/wasmtime-go>
- bytecodealliance. (2023, April 7). *componentize-py*. componentize-py. Retrieved March 16, 2026, from <https://github.com/bytecodealliance/componentize-py>
- BytecodeAlliance. (2025, November 25). *WebAssembly Micro Runtime*. Github. <https://github.com/bytecodealliance/wasm-micro-runtime>
- Byteiota. (2026, March 7). *WASI 0.3 Native Async: WebAssembly Gets Concurrent I/O*. WASI 0.3 Native Async: WebAssembly Gets Concurrent I/O. <https://byteiota.com/wasi-0-3-native-async-webassembly-gets-concurrent-i-o/>
- Chiarlone, D. (2025). *Server-Side WebAssembly: Building Portable and Secure Multi-language Apps*. Manning.
- Chicory. (2026). *Chicory, a JVM native WebAssembly runtime*. Chicory.dev. <https://chicory.dev/>
- Chudasama, Y. (2026, February 28). *WebAssembly Beyond the Browser: The Universal Runtime*. [yashchudasama.com. https://www.yashchudasama.com/blog/tech/wasm-beyond-browser/](https://www.yashchudasama.com/blog/tech/wasm-beyond-browser/)
- Cloudflare Inc. (2026, January 26). *Overview · Cloudflare Workers docs*. Cloudflare. Retrieved March 17, 2026, from <https://developers.cloudflare.com/workers/>
- Deno. (n.d.). *Deno*. Deno, the next-generation JavaScript runtime. Retrieved February 24, 2026, from <https://deno.com/>
- Docker. (2022, October 19). *Wasm workloads*. Docker Docs. Retrieved February 24, 2026, from <https://docs.docker.com/desktop/features/wasm/>

Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., & Bastien, J. (2017). Bringing the Web up to Speed with WebAssembly. (Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)), 185-200.  
<https://doi.org/10.1145/3062341.3062363>

Has, M. (2025, December 12). *WebAssembly on Tiny IoT Devices: How Far Can We Push Portability*. AIoTwin.  
[https://aiotwin.eu/aiotwin/results/blog/webassembly\\_on\\_tiny\\_iot\\_devices\\_how\\_far\\_can\\_we\\_push\\_portability](https://aiotwin.eu/aiotwin/results/blog/webassembly_on_tiny_iot_devices_how_far_can_we_push_portability)

*A high-performance embeddable WebAssembly runtime for the JVM*. (n.d.). GraalVM. Retrieved March 10, 2026, from <https://www.graalvm.org/webassembly/>

k3d. (2024, January 2). *k3d overview*. k3d. <https://k3d.io/stable/>

KodeKloud. (n.d.). *Wasm Runtimes*. KodeKloud. Retrieved March 12, 2026, from <https://notes.kodekloud.com/docs/Exploring-WebAssembly-WASM/Future-WebAssembly-in-Cloud/WASM-Runtimes/page>

KodeKloud. (2023, February 20). *WebAssembly (WASM) vs. Docker - Our Expert Analysis [VIDEO]*. Youtube.  
<https://www.youtube.com/watch?v=7553XZ0T6pM>

Krustlet. (2023, October 2). *Krustlet: Kubernetes Kubelet in Rust for running WASM*. Github.  
<https://github.com/krustlet/krustlet/>

lancemcluskey, & Wunder, C. (n.d.). *Node.js with WebAssembly*. Node.js. Retrieved February 23, 2026, from <https://nodejs.org/en/learn/getting-started/nodejs-with-webassembly>

Lee, H. (2026, Feb 10). *WebAssembly Beyond the Browser: WASI 2.0, the Component Model, and Why Wasm Is About to Change Everything*.  
[webassembly-beyond-the-browser-wasi-20-the-component-model-and-why-wasm-is-about-to-change.](https://dev.to/pocket_tools/webassembly-beyond-the-browser-wasi-20-the-component-model-and-why-wasm-is-about-to-change-3ep0)  
[https://dev.to/pocket\\_tools/webassembly-beyond-the-browser-wasi-20-the-component-model-and-why-wasm-is-about-to-change-3ep0](https://dev.to/pocket_tools/webassembly-beyond-the-browser-wasi-20-the-component-model-and-why-wasm-is-about-to-change-3ep0)

Mears, J. (2019, June 20). *How we're bringing Google Earth to the web*. web.dev.  
<https://web.dev/case-studies/earth-webassembly>

Murphy, C. (2022, Jul 30). *Server-side Wasm Applications: Replacing Docker with WASI*. Server-side Wasm Applications: Replacing Docker with WASI - Colin Murphy, Adobe.  
[https://www.youtube.com/watch?v=KGNNE\\_imU5g&t=2818s](https://www.youtube.com/watch?v=KGNNE_imU5g&t=2818s)

*Photoshop's journey to the web | Articles*. (2021, October 26). web.dev. Retrieved March 12, 2026, from <https://web.dev/articles/ps-on-the-web>

Python Software Foundation. (2017, February 11). *CPython*. CPython. Retrieved March 16, 2026, from <https://github.com/python/cpython/tree/main>

Titizer, B. L. (n.d.). *The Wizard Research Engine*. Github. Retrieved February 24, 2026, from <https://github.com/titizer/wizard-engine>

Uwaila, G. (2026, March 3). *Test WASI API — Language Comparison*. experiments README.md README.md. [https://gitlab.com/kdg-ti/the-lab/teams-25-26/44\\_theinkerers/experiments/-/blob/main/README.md](https://gitlab.com/kdg-ti/the-lab/teams-25-26/44_theinkerers/experiments/-/blob/main/README.md)

vshymanskyi. (2021, June 2). *Wasm3 - A fast WebAssembly interpreter and the most universal WASM runtime*. Github. <https://github.com/wasm3/wasm3>

WABT: *The WebAssembly Binary Toolkit*. (2015, September 7). WABT: The WebAssembly Binary Toolkit. Retrieved march 16, 2026, from <https://github.com/WebAssembly/wabt>

WASI. (n.d.). *Roadmap*. WASI.dev. Retrieved March 16, 2026, from <https://wasi.dev/roadmap>

WASI Subgroup. (n.d.). *WASI Introduction*. WASI.dev. Retrieved March 12, 2026, from <https://wasi.dev/>

WASI Subgroup. (n.d.). *WebAssembly/WASI: WebAssembly System Interface · GitHub*. GitHub. Retrieved March 12, 2026, from <https://github.com/WebAssembly/WASI>

WasmEdge. (n.d.). *UDF in the libSQL database*. wasmedge.org. <https://wasmedge.org/docs/embed/use-case/libsql/>

WasmEdge. (n.d.). *WasmEdge: Bring the cloud-native and serverless application paradigms to Edge Computing*. WasmEdge. Retrieved February 6, 2026, from <https://wasmedge.org/>

Wasmer: *Universal applications using WebAssembly*. (n.d.). Wasmer. Retrieved February 6, 2026, from <https://wasmer.io/>

WasmIO. (2025, May 14). *Running AI workloads with WASM - is it production ready? [Panel] @ Wasm I/O 2025*. Youtube WasmIO. <https://www.youtube.com/watch?v=UODo4Q5ykd8&t=747s>

wasmruntime. (n.d.). *All Runtimes*. All Runtimes. Retrieved March 12, 2026, from <https://wasmruntime.com/en#authority-matrix>

Wazero. (n.d.). *Wazero: the zero dependency WebAssembly runtime for Go developers*. wazero. Retrieved February 24, 2026, from <https://wazero.io/>

webassembly. (n.d.). *Feature Status*. features. Retrieved March 12, 2026, from <https://webassembly.org/features/?categories=standalones>

World Wide Web Consortium. (2019, December 5). *WebAssembly Core Specification*. WebAssembly Core Specification. Retrieved march 16, 2026, from <https://www.w3.org/TR/wasm-core-1/>

Wright, E. (2025, Dec 3). *Akamai's Edge Strategy Grows Again: KubeCon 2025 + Fermion Acquisition*. Cloud Akamai's Edge Strategy Grows Again: KubeCon 2025 +. <https://techforward.io/akamais-edge-strategy-grows-kubecon-2025-fermyon-acquisition/>